Programación II

Guía Práctica N.º 9

Librerías aplicadas en Ingeniería

Clase 3: de las mediciones a la información

Lenguaje: Python
Librerías: Pandas, NumPy y Matplotlib
Entorno: Visual Studio Code y `.venv`

Edición 2026

Índice

1 Presentación y propósito **1**

2 Indicaciones generales de trabajo **1**

3 Ejercicios obligatorios **2**

1. Inventario de la red . . . 2
2. Calidad de los datos . . . 3
3. Clasificación de la calidad de señal . . . 3
4. Perfil horario de la red . . . 3
5. Resumen diario de la red . . . 3

4 Ejercicios propuestos **4**

6. Pérdida de trayecto y distancia equivalente . . . 4
7. Ranking de temperatura . . . 4
8. ¿La temperatura afecta la señal? . . . 5
9. Exportar e importar un informe . . . 5
10. Cada estación contra la red . . . 5
11. Disponibilidad de cada enlace . . . 5
12. Detección automática de huecos en toda la red . . . 6
13. Detectar degradaciones por la pendiente . . . 6
14. Alarmas persistentes . . . 7
15. Análisis de un evento de sobretemperatura . . . 7
16. Comparación por banda de frecuencia . . . 7
17. Simular una estación nueva . . . 7
18. Tablero de la red . . . 8
19. ¿Cuándo va a dejar de funcionar? . . . 8
20. Integrador: informe de mantenimiento priorizado . . . 8

5 Lista de comprobación final **9**

1 Presentación y propósito

Esta guía acompaña la tercera clase de la unidad **Librerías aplicadas en Ingeniería**. Se analizará la telemetría de una red de 20 estaciones mediante Pandas, en combinación con NumPy y Matplotlib, como en las actividades de un centro de operación de una red de telecomunicaciones.

Es la **Guía Práctica N.º 9** de Programación II y conserva la numeración general de las guías anteriores de C, C++ y Python.

Resultados de aprendizaje

Al finalizar las actividades, se espera que puedas:

- importar y explorar registros tabulares de telemetría usando Pandas;
- agrupar mediciones por estación, fecha, hora y frecuencia de operación;
- detectar datos faltantes, alarmas, períodos sin reportes y degradaciones;
- integrar Pandas con NumPy y Matplotlib para realizar cálculos y presentar resultados;
- elaborar informes técnicos a partir de los indicadores de una red;
- organizar las soluciones con funciones documentadas y una función `main()`.

2 Indicaciones generales de trabajo

- **Ejercicios 1 a 5: obligatorios.** Se desarrollarán a partir de lo presentado en la tercera clase de Python.
- **Ejercicios 6 a 20: propuestos.** Son optativos; se recomienda resolverlos, porque amplían los temas y tienen complejidad creciente.
- Todos los ejercicios emplean, salvo indicación contraria, el archivo `telemetria.csv` provisto por la cátedra. Ubicalo en la misma carpeta que el archivo `.py`.
- Cada tarea se resuelve mediante funciones con su *docstring* (`"""..."""`), y una función `main()` llamada al final del archivo, siguiendo las convenciones de esta unidad. No es necesario definir clases.
- Los valores fijos (umbrales, pesos y constantes físicas) se definen en mayúsculas al comienzo del programa.
- Los gráficos deben tener título, ejes con unidades, grilla y leyenda cuando corresponda.
- Algunos ejercicios piden consultar métodos de Pandas en la documentación oficial (https://pandas.pydata.org/docs/); la consulta forma parte de la actividad.
- Todos los archivos de esta clase pueden guardarse dentro de `GuiaPython/Clase3`; la instalación del entorno se realiza según la **Guía 0 — Ampliación para Python**.

Conceptos en caja negra

Cuando aparezcan conceptos todavía no abordados en otras asignaturas, se proporciona una definición sencilla, la ecuación correspondiente y la referencia a la materia donde se estudiarán con mayor profundidad. El objetivo aquí es aplicar correctamente las expresiones, sus unidades y el código.

Antes de comenzar: importá las bibliotecas necesarias, incluida Pandas con su alias convencional `pd`, y leé el archivo de la siguiente forma:

```
import pandas as pd

df = pd.read_csv("telemetria.csv", parse_dates=["timestamp"])
```

El archivo tiene las siguientes columnas:

Columna	Unidad	Descripción
`timestamp`	fecha y hora	Momento del reporte (cada 15 minutos).
`estacion`	—	Identificador de la estación (E01 a E20).
`frecuencia`	MHz	Banda de operación del enlace.
`potencia`	dBm	Potencia de transmisión.
`rssi`	dBm	Potencia recibida.
`temperatura`	°C	Temperatura del equipo.

Umbrales de alarma de la unidad: RSSI < -75 dBm y temperatura > 60 °C. Los promedios de RSSI se calculan en **potencia**, como se trabajó en la caja negra de la Clase 1.

Glosario rápido:

DataFrame: tabla de Pandas que organiza registros en filas y columnas.

RSSI: indicador de intensidad de señal recibida, expresado aquí en dBm.

`timestamp`: fecha y hora asociadas a cada reporte.

Dato faltante: valor ausente, identificable mediante `isna()`.

Alarma de RSSI: medición con RSSI inferior a -75 dBm.

Alarma de temperatura: medición superior a 60 °C.

3 Ejercicios obligatorios

1. Inventario de la red

Antes de analizar mediciones, un ingeniero necesita saber qué equipos tiene. Cada estación opera siempre en la misma banda y con la misma potencia de transmisión.

1. Construí una tabla con una fila por estación y sus columnas `frecuencia` y `potencia`. Sugerencia: `groupby("estacion")` y el método `first()`, que toma el primer valor de cada grupo.
2. Comprobá con `nunique()` que ninguna estación haya cambiado de frecuencia o potencia durante la semana.
3. Contá cuántas estaciones operan en cada banda y calculá la potencia de transmisión promedio de cada banda.
4. Agregá a la tabla una columna con la longitud de onda en centímetros de cada estación, empleando $\lambda = c/f$.

2. Calidad de los datos

Un análisis confiable empieza por saber cuántos datos faltan y dónde.

1. Calculá, para cada estación, la cantidad de valores faltantes de `rssi` y `temperatura`. Sugerencia: crear una columna booleana con `isna()` y sumarla con `groupby()`.
2. Calculá el porcentaje de datos faltantes de cada columna en toda la red.
3. ¿Qué estación tiene más datos faltantes de RSSI? ¿Hay alguna que no tenga ninguno?
4. Graficá con barras la cantidad de faltantes de RSSI por estación.

¿Los faltantes están concentrados en pocas estaciones o repartidos? ¿Qué implica eso para la decisión de descartarlos?

3. Clasificación de la calidad de señal

Un operador clasifica cada medición según su RSSI:

Calidad	**Rango de RSSI**
Excelente	RSSI ≥ -60 dBm
Buena	-67 dBm $\leq$ RSSI < -60 dBm
Regular	-75 dBm $\leq$ RSSI < -67 dBm
Mala	RSSI < -75 dBm

1. Agregá al DataFrame una columna `calidad`. Podés hacerlo con `np.select()` o con `pd.cut()` (buscá en la documentación el parámetro `right`, necesario para respetar exactamente los límites de la tabla).
2. Contá cuántas mediciones de toda la red hay en cada categoría y el porcentaje que representan.
3. Para cada estación, calculá el porcentaje de mediciones "Excelente" y ordená las estaciones de mayor a menor.
4. Verificá que la cantidad de mediciones "Mala" coincide con la cantidad total de alarmas de RSSI.

4. Perfil horario de la red

¿Cambia el comportamiento de la red según la hora del día?

1. Usando `df["timestamp"].dt.hour`, calculá para cada hora (0 a 23): la temperatura promedio de toda la red y el RSSI promedio en potencia.
2. Graficá los dos perfiles en una figura con `plt.subplot(2, 1, ...)`, con la hora en el eje X.
3. Informá la hora más caliente y la más fría (`idxmax()`, `idxmin()`) y la diferencia de temperatura entre ambas.

¿El RSSI muestra un ciclo diario como la temperatura? ¿Qué conclusión sacás?

5. Resumen diario de la red

El centro de operación recibe cada mañana un resumen del día anterior. Usando `df["timestamp"].dt.date`, construí una tabla con una fila por día y estas columnas:

- RSSI promedio de la red, en potencia;
- temperatura máxima registrada y la estación que la registró;
- cantidad de alarmas de RSSI;
- cantidad de alarmas de temperatura;
- cantidad de reportes recibidos (se esperan $20 \times 96 = 1920$ por día).

Mostrá la tabla y graficá con barras las alarmas de RSSI por día. ¿Qué días fueron problemáticos? ¿Qué muestra la cantidad de reportes?

4 Ejercicios propuestos

6. Pérdida de trayecto y distancia equivalente

Con la potencia de transmisión y el RSSI se puede estimar cuánto se pierde en el trayecto de cada enlace. Suponé antenas de 2 dBi en ambos extremos.

CAJA NEGRA — Pérdida de trayecto y distancia equivalente

Qué es: la pérdida de trayecto es todo lo que se pierde entre transmisor y receptor. La distancia equivalente en espacio libre es la distancia a la que la FSPL sería igual a esa pérdida: una forma de comparar enlaces de distintas bandas.
Cómo lo usamos:

$$L_{\text{trayecto}}\,[\text{dB}] = P_{\text{tx}} + G_{\text{tx}} + G_{\text{rx}} - \text{RSSI}_{\text{promedio}},$$

$$d_{\text{eq}}\,[\text{m}] = \frac{c}{4\pi f}\, 10^{L_{\text{trayecto}}/20}, \qquad f \text{ en Hz}.$$

Lo van a profundizar en: Antenas y Propagación.

Calculá, para cada estación, el RSSI promedio en potencia, la pérdida de trayecto y la distancia equivalente en km. Mostrá la tabla ordenada por pérdida. ¿Qué estación tiene la mayor pérdida? ¿Coincide con la de mayor distancia equivalente? ¿Por qué no necesariamente?

7. Ranking de temperatura

1. Calculá por estación la temperatura mínima, promedio y máxima (`agg()`).
2. Listá las 5 estaciones con mayor temperatura máxima.
3. Calculá para cada estación la amplitud térmica (máxima − mínima) y encontrá la de mayor amplitud.
4. Calculá cuántas estaciones superaron los 35 °C al menos una vez.

¿Hay una estación cuya temperatura máxima se aparte claramente del resto?

8. ¿La temperatura afecta la señal?

CAJA NEGRA — Coeficiente de correlación

Qué es: un número entre -1 y 1 que mide si dos magnitudes varían juntas. Cerca de 1: cuando una sube, la otra sube. Cerca de -1: cuando una sube, la otra baja. Cerca de 0: no hay relación lineal.
Cómo lo usamos: `df["rssi"].corr(df["temperatura"])`.
Lo van a profundizar en: Probabilidad y Estadística.

1. Calculá la correlación entre RSSI y temperatura en toda la red.
2. Calculá la correlación para cada estación y mostrala ordenada.
3. Para la estación de correlación más alta (en valor absoluto), graficá un diagrama de dispersión (`plt.scatter()`) de temperatura contra RSSI.

¿Hay evidencia de que la temperatura afecte el RSSI en esta red? ¿Qué valores de correlación esperarías si, por ejemplo, un amplificador perdiera ganancia al calentarse?

9. Exportar e importar un informe

1. Construí una tabla resumen por estación con: banda, RSSI promedio en potencia, RSSI mínimo, cantidad de alarmas de RSSI (0 si no tuvo) y temperatura máxima. Redondeá a 2 decimales.
2. Guardala en `resumen_red.csv` con `to_csv()`.
3. Buscá en la documentación cómo guardar la misma tabla en un archivo separado por punto y coma (`;`), que es el formato que abre directamente una planilla de cálculo configurada en español. Guardala como `resumen_red_es.csv`.
4. Leé los dos archivos con `pd.read_csv()` (con el separador adecuado) y verificá que las dos tablas leídas son iguales (buscá el método `equals()`).

10. Cada estación contra la red

Una estación puede tener un RSSI "normal" en valor absoluto, pero comportarse distinto del resto de la red.

1. Para cada instante (`timestamp`), calculá la mediana del RSSI de todas las estaciones.
2. Para cada medición, calculá la diferencia entre su RSSI y la mediana de la red en ese instante. Sugerencia: buscá en la documentación `groupby(...).transform("median")`, que devuelve el resultado del grupo alineado con cada fila.
3. Calculá por estación el promedio de esa diferencia al comienzo de la semana (5 y 6 de octubre) y al final (10 y 11 de octubre).
4. ¿Qué estación cambió más su posición relativa respecto de la red?

11. Disponibilidad de cada enlace

La disponibilidad es el porcentaje del tiempo en que un enlace funciona. Un reporte que no llegó también cuenta como tiempo no disponible.

1. Calculá la cantidad de reportes esperados por estación en la semana (7×96 reportes).
2. Para cada estación, contá los reportes con RSSI válido y por encima del umbral.
3. Calculá la disponibilidad de cada estación:

$$\text{disponibilidad (en porcentaje)} = \frac{\text{reportes válidos y sin alarma}}{\text{reportes esperados}}\,100.$$

4. Un contrato de servicio exige 99% de disponibilidad. ¿Qué estaciones no lo cumplen y por qué motivo (alarmas, reportes faltantes o datos faltantes)?

12. Detección automática de huecos en toda la red

Generalizá el ejercicio "Estación silenciosa" de la clase: escribí la función

```
detectar_huecos(df, intervalo_min)
```

La función debe recorrer todas las estaciones y devolver un DataFrame con una fila por hueco encontrado y las columnas:

```
estacion, ultimo_reporte, siguiente_reporte,
duracion, reportes_faltantes
```

Probala con intervalos de 15 minutos. Luego, para probar que funciona con varios huecos, creá una copia de los datos en la que elimines los registros de E05 del 6/10 entre las 08:00 y las 09:45 y los de E09 del 10/10 entre las 20:00 y las 20:30, y verificá que los detecta.

13. Detectar degradaciones por la pendiente

Una degradación lenta puede detectarse antes de que dispare alarmas, midiendo cuánto cae el RSSI por día.

CAJA NEGRA — Recta de ajuste (regresión lineal)

Qué es: la recta que mejor se aproxima a un conjunto de puntos. Su pendiente indica cuánto varía y por cada unidad de x.
Cómo lo usamos: `pendiente, ordenada = np.polyfit(x, y, 1)`, con `x` e `y` como arrays numéricos sin valores NaN.
Lo van a profundizar en: Probabilidad y Estadística y Métodos Numéricos.

1. Para cada estación, descartá los NaN de RSSI y calculá `x` como los días transcurridos desde el primer reporte:

```
x = (g["timestamp"] - g["timestamp"].min()).dt.total_seconds() / 86400
```

2. Calculá la pendiente del RSSI en dB/día de cada estación.
3. Marcá como "EN DEGRADACIÓN" las estaciones con pendiente menor que -1 dB/día.
4. Para la estación detectada, graficá el RSSI y la recta de ajuste.

¿Qué estaciones tienen pendiente positiva? ¿Es preocupante una pendiente de $\pm 0{,}05$ dB/día?

14. Alarmas persistentes

Una muestra aislada bajo el umbral puede ser un desvanecimiento; una hora entera bajo el umbral es un problema real. El operador quiere una alarma persistente: 4 o más muestras consecutivas por debajo de -75 dBm (una hora).

1. Para cada estación, creá una serie booleana de alarma y aplicale `rolling(4).sum()`: donde el resultado sea 4, hay 4 alarmas seguidas.
2. Contá, por estación, cuántas veces se cumple la condición de alarma persistente.
3. Informá la fecha y hora de la primera alarma persistente de cada estación que la tenga.
4. Compará con la cantidad de alarmas simples. ¿Qué estaciones desaparecen con este criterio? ¿Por qué el criterio es útil para un centro de operación?

15. Análisis de un evento de sobretemperatura

1. Encontrá la estación y el momento de la temperatura máxima de toda la semana.
2. Extraé los registros de esa estación desde 3 horas antes hasta 3 horas después de ese momento (usá `pd.Timedelta(hours=3)` y dos condiciones sobre `timestamp`).
3. Calculá la duración del evento: tiempo durante el cual la temperatura superó 60 °C (cantidad de muestras $\times$ 15 minutos).
4. Calculá cuánto por encima de su temperatura habitual para esa hora estuvo la estación: compará el máximo del evento con el promedio de la misma estación, a la misma hora, en los otros días.
5. Graficá la temperatura en la ventana de 6 horas con la línea de 60 °C y el RSSI en un segundo gráfico (`subplot`). ¿El RSSI se vio afectado?

16. Comparación por banda de frecuencia

1. Agregá la banda a cada medición (ya está en `frecuencia`) y calculá por banda: RSSI promedio en potencia, desvío del RSSI y cantidad de alarmas.
2. Buscá en la documentación de Matplotlib la función `plt.boxplot()` y graficá la distribución del RSSI de cada banda en un mismo gráfico (una "caja" por banda). Descartá los NaN antes.
3. Leé en la documentación qué representan la caja, la línea central y los puntos sueltos del diagrama.

¿Alguna banda se comporta distinto? ¿Los puntos sueltos pertenecen a alguna estación en particular?

17. Simular una estación nueva

La red va a incorporar la estación E21 (2400 MHz, 23 dBm). Antes de instalarla, se quiere probar que los programas de análisis la procesan bien.

1. Buscá en la documentación `pd.date_range()` y creá los 672 instantes de la semana, cada 15 minutos, desde el 2026-10-05 00:00.
2. Con `rng = np.random.default_rng(21)`, simulá su RSSI con media -66 dBm y desvío 1,5 dB, y su temperatura como

$$T = 25 + 7\,\mathrm{sen}\left(\frac{2\pi(\mathrm{hora} - 9)}{24}\right)$$

más ruido de desvío 0,8 (hora: `dt.hour` de cada instante).

3. Armá un DataFrame con las mismas columnas que `telemetria.csv` y unilo al original con `pd.concat()` (buscalo en la documentación).
4. Verificá que la red ahora tiene 21 estaciones y 14 072 registros, y repetí el cálculo del margen por estación.

18. Tablero de la red

Construí un tablero (*dashboard*): una única figura de 2×2 (`figsize=(14, 9)`) que resuma el estado de la red:

1. arriba a la izquierda: margen promedio sobre el umbral por estación (barras), con las estaciones de menos de 10 dB de margen en otro color;
2. arriba a la derecha: alarmas de RSSI por día (barras);
3. abajo a la izquierda: RSSI y promedio móvil de la estación con más alarmas;
4. abajo a la derecha: perfil horario de temperatura de la red y de la estación más caliente.

Guardá el tablero como `tablero_red.png` con `dpi=150`. Cada gráfico debe tener título y ejes con unidades.

19. ¿Cuándo va a dejar de funcionar?

La estación en degradación que detectaste en el ejercicio 13 sigue cayendo. Por debajo de -85 dBm el enlace se corta por completo.

1. Tomá los datos de esa estación desde el inicio de la degradación (determinalo mirando el gráfico del promedio móvil).
2. Calculá la recta de ajuste de esos datos, con `x` en días desde el 2026-10-05 00:00.
3. Despejá de la recta en qué momento el RSSI promedio alcanzaría -85 dBm:

$$x = \frac{-85 - \text{ordenada}}{\text{pendiente}}.$$

   Convertilo en fecha y hora.
4. Graficá los datos, la recta extendida hasta ese momento y las líneas de -75 y -85 dBm.

¿Cuánto tiempo tiene el equipo de mantenimiento? ¿Qué tan confiable es esta predicción? Mencioná al menos dos motivos por los que podría fallar.

20. Integrador: informe de mantenimiento priorizado

El jefe de mantenimiento tiene un solo técnico disponible y necesita saber a qué estaciones enviarlo primero. Escribí un programa que calcule, para cada estación, un puntaje de prioridad que combine varios indicadores:

Indicador	Puntos
Cada alarma persistente de RSSI (ejercicio 14)	2
Cada alarma simple de RSSI	0,5
Pendiente de RSSI menor que -1 dB/día (ejercicio 13)	50
Cada alarma de temperatura	5
Cada reporte faltante (ejercicio 12)	1
Margen promedio menor que 10 dB	10

El programa debe:

1. calcular cada indicador con una función propia, reutilizando las de los ejercicios anteriores;
2. reunir los indicadores en un DataFrame y calcular el puntaje total;
3. ordenar las estaciones por puntaje y asignar prioridad “ALTA” (puntaje ≥ 50), “MEDIA” (entre 10 y 50) o “BAJA” (menos de 10);
4. generar un archivo `informe_mantenimiento.md` con una tabla de las estaciones de prioridad ALTA y MEDIA, y para cada una un párrafo con el diagnóstico (qué indicador pesó más) y una acción recomendada (por ejemplo: “verificar alineación de antena”, “revisar ventilación”, “revisar alimentación y comunicaciones”);
5. guardar un gráfico de barras horizontales con el puntaje de cada estación.

Los pesos de la tabla deben definirse como constantes, de modo que el jefe de mantenimiento pueda ajustarlos sin modificar las funciones.

5 Lista de comprobación final

Antes de dar por terminada la guía

- ☐ Se utilizó el entorno Python de la unidad y las librerías necesarias.
- ☐ Se trabajó con `telemetria.csv` en la carpeta indicada.
- ☐ Los programas se organizaron en funciones documentadas y una llamada a `main()`.
- ☐ Se distinguieron los valores faltantes de los reportes ausentes.
- ☐ Se aplicaron correctamente los umbrales de RSSI y temperatura.
- ☐ Los promedios de RSSI se calcularon en potencia.
- ☐ Las operaciones de agrupamiento respetan estaciones, fechas y horas.
- ☐ Los gráficos cuentan con títulos, ejes con unidades y leyendas cuando corresponde.
- ☐ Los archivos generados y las conclusiones coinciden con las consignas.
- ☐ Se respondieron las preguntas de interpretación de los ejercicios desarrollados.